

TDT4145 – DATAMODELLERING OG DATABASESYSTEMER

Del 2

Gruppe 210: Eirin Husby Iversen

I del 2 av prosjektet har vi realisert databasesystemet for SiT Trening ved bruk av Python og SQLite. Systemet fungerer som en løsning for administrasjon av treningssentre, saler, utstyr, samt håndtering av booking m.m. Realiseringen bygger direkte på relasjonsdatabaseskjemaet fra DB1. Et overordnet mål i implementasjonen har vært å prioritere SQL framfor Python for å løse forretningslogikk der det er mulig. Dette sikrer at databasen håndterer integritet og restriksjoner, noe som resulterer i en mer robust implementasjon.

Relasjonsdatabaseskjemaet fra del 1 er implementert, men det er gjort én strategisk endring for å sikre at systemet er etterprøvbart og kan reproduseres. Vi har introdusert en ny tabell som vi har valgt å kalle *system_tid* som er av typen DATETIME, som lagrer en «simulert nåtid». Oppgaven krever at programmet skal kunne kjøres uten å streve, og at resultatene skal kunne reproduseres av sensor i framtiden. Siden flere av systemets triggere er tidsavhengige, som 48-timersregelen for booking, ville bruk av reell systemtid gjort det umulig å teste disse reglene mot de forutbestemte testdataene. Ved å bruke simulert tid kan vi sikre at logikken fungerer som tiltenkt uavhengig av når sensuren måtte skje.

Utover dette har vi implementert de komplekse restriksjonene som ble dokumentert i DB1 gjennom SQL-triggere. Disse triggerne håndterer dynamiske operasjoner som kø-logikk, integritet ved booking og tidsfrister, og sikrer at systemet overholder integritetsreglene fra oppgavebeskrivelsen. Det som derimot ikke er implementert og som ble nevnt i DB1 at måtte implementeres i applikasjonen, er den dynamiske statusoppdateringen. Denne oppdateringen skulle gjøre at status på en booking automatisk ble satt til 'Ikke møtt' 5 minutter før gruppetimen startet hvis oppmøte ikke var registrert. Denne løsningen skulle løses i Python, men vi har valgt å ikke implementere det da det ikke inngår direkte i noen av brukstilfellene, som var hovedoppgaven å løse for DB2.

Implementasjon

Systemet er implementert som en tekstbasert applikasjon i Python, som kommuniserer med en SQLite-database ved hjelp av sqlite3-modulen. Vi har lagt stor vekt på en oversiktlig filstruktur for å gjøre det enkelt for sensor å navigere i kildekoden. Kildekoden er delt inn i følgende:

- **trening.sql**: Inneholder det komplette relasjonsskjemaet slik det ble presentert i DB1, i tillegg til tabellen `system_tid` for å håndtere simulert nåtid
- **triggere.sql**: En egen fil dedikert til alle databasens triggere som brukes for å opprettholde integritetsregler.
- **testdata.sql**: Et SQL-skript som legger inn testdata som dekker alle brukstilfellene og gjør det mulig å teste systemet direkte.
- **main.py**: Selve kjøringen av programmet som gir et enkelt tekstbasert grensesnitt for å kjøre de ulike brukstilfellene.
- **brukstilfelleX.py**: Hvert av de åtte brukstilfellene har sin egen dedikerte Python-fil. Disse filene inneholder implementasjonen av hvert brukstilfelle, bestående av SQL-spøringer og eventuell Python-logikk for håndtering av input og output.

For hvert brukstilfelle har vi strukturert koden slik at selve løsningen er tydelig kommentert. Dette gjør det lett å identifisere kjerne-logikken, enten den består av SQL-spøringer eller en Python-funksjon.

Selv om oppgaven spesifiserer at enkelte brukstilfeller kan løses i ren SQL, har vi valgt å inkludere Python-kode for samtlige tilfeller. Dette er gjort for å demonstrere at løsningene fungerer som tiltenkt ved å skrive ut resultatene til terminalen.

I implementasjonen har vi fulgt følgende praksiser:

- Vi har flyttet mest mulig logikk til SQL-triggere for å ivareta dataintegriteten direkte i databaselaget
- Ved overføring av parametere fra Python til SQL har vi brukt plassholdere ``(?)`` i spørringer for å bruke gitte parametere
- Vi benytter ``commit()`` for å sikre at endringer lagres korrekt, og sikrer at fremmednøkler alltid er aktivert med ``PRAGMA foreign_keys = ON``

b) Oppskrift på kjøring av programmet

Under er en oppskrift på hvordan databaseapplikasjonen kan kjøres:

1. **Åpne et terminalvindu**

2. **Naviger til prosjektmappen `TDT4145-Prosjekt`**

```
cd /TDT4145-Prosjekt
```

Du er i riktig mappe når du står i samme mappe som `main.py` og de andre filene.

3. **Sørg for at Python 3 er installert**

4. **Kjør `main.py` for å starte programmet med følgende kommando:**

```
python3 main.py
```

Når programmet starter, settes databasen opp ved å kjøre SQL-skriptene kjøres. Dette oppretter tabeller, triggerer og legger inn testdata, slik at systemet er klart til bruk.

Databasen initialiseres på nytt ved hver kjøring.

5. **Kjør de forskjellige brukstilfellene ved å følge instruksjonene i menyen som vises i terminalen.**

Merk at for å kjøre brukstilfelle 3 og faktisk registrere oppmøte, må du først ha kjørt brukstilfelle 2 for å booke en gruppetime.

*For å gjøre det lett å kjøre de forskjellige brukstilfellene med standardverdier, er det mulig å trykke **Enter** isteden for å skrive inn verdier manuelt. Disse standardverdiene er valgt slik at de samsvarer med dataene som er oppgitt i beskrivelsene av brukstilfellene.*

c) De tekstlige resultatene

Under vises de tekstlige resultatene (output) fra kjøring av de ulike brukstilfellene. Output er etterprøvbart og kan enkelt reproduseres ved å følge oppskriften i del b), og kjøre de forskjellige brukstilfellene i menyen med standardverdiene for input (trykke **Enter** for alle parametere).

Brukstilfelle 1:

'Legg inn treningssenter, saler, noen sykler, noen brukere, noen trenere og treninger som nevnt over. Dette skal leveres som SQL.'

Dette brukstilfellet initialiserer databasen og legger inn nødvendige testdata.

OUTPUT:

```
Brukstilfelle 1: Initialiserer og legger til data i database...
Suksess: Database er satt opp med tabeller, triggere og testdata!
```

```
----- TRENING DB -----
Brukstilfeller:
2. Booking av gruppetime
3. Registrer oppmøte for gruppetime
4. Ukeplan for en gitt uke
5. Personlig besøkshistorie for en gitt bruker
6. Simulering av svartelisting
7. Flest deltakelser for en gitt måned
8. Finn treningspartnere
0. Avslutt
```

```
=====
Velg et brukstilfelle (2-8, eller 0 for å avslutte):
```

Brukstilfelle 2:

'Booking av trening «Spin60» på tirsdag 17. mars kl. 18.30 på Øya treningssenter for bruker «johnny@stud.ntnu.no». Denne skal leveres som både Python og SQL. La brukernavn, aktivitet og tidspunkt være parametere, og sjekk at treningen finnes før dere booker.'

Dette brukstilfellet demonstrerer booking av en gruppetime med gitte parametere.

OUTPUT:

```
Velg et brukstilfelle (2-8, eller 0 for å avslutte): 2
```

```
BOOKING AV GRUPPETIME
```

```
-----
Oppgi følgende informasjon for å booke en gruppetime:
E-post (trykk Enter for johnny@stud.ntnu.no):
Aktivitet (trykk Enter for Spin60):
Starttid (YYYY-MM-DD HH:MM:SS, trykk Enter for 2026-03-17 18:30:00):
```

```
Suksess! Booking opprettet for johnny@stud.ntnu.no på Spin60.
```

Brukstilfelle 3:

'Registrering av oppmøte for treningen nevnt i brukstilfelle 2. Brukernavn og hvilken trening skal være parametere. Denne skal leveres som Python og SQL.'

Dette brukstilfellet demonstrerer registrering av oppmøte.

Dersom brukstilfelle 3 kjøres før brukstilfelle 2, vil man derimot ikke kunne registrere oppmøte:

```
Velg et brukstilfelle (2-8, eller 0 for å avslutte): 3
REGISTER OPPMØTE FOR GRUPPETIME
-----
Oppgi e-post for å finne planlagte bookinger og registrere oppmøte:
E-post (trykk Enter for johnny@stud.ntnu.no):
Ingen bookinger funnet for johnny@stud.ntnu.no i dag.
```

Hvis brukstilfelle 2 kjøres først slik at bookingen er opprettet, vil brukstilfelle 3 kunne registrere oppmøte for den bookingen.

OUTPUT:

```
Velg et brukstilfelle (2-8, eller 0 for å avslutte): 3
REGISTER OPPMØTE FOR GRUPPETIME
-----
Oppgi e-post for å finne planlagte bookinger og registrere oppmøte:
E-post (trykk Enter for johnny@stud.ntnu.no):
Suksess! Oppmøte registrert for johnny@stud.ntnu.no på Spin60.
```

Brukstilfelle 4:

'Ukeplan for alle treninger registrert i uke 12, dvs. fra 16.mars til 23.mars. Denne skal sorteres på tid, dvs. treninger fra forskjellige senter skal flettes inn i samme output. Dette skal leveres i Python og SQL. Startdag og uke skal være parametere som settes før du kjører queryet.'

Dette brukstilfellet viser en samlet ukeplan sortert på tidspunkt.

OUTPUT:

```

Velg et brukstilfelle (2-8, eller 0 for å avslutte): 4

UKEPLAN FOR EN GITT UKE
-----
Oppgi ukenummer for å hente ukeplan:
Skriv inn ukenummer (1-52, trykk Enter for 12):

UKEPLAN FOR UKE 12 (Starter 2026-03-16, Slutter 2026-03-23)
-----

```

Tid	Aktivitet	Senter	Sal	Type
2026-03-16 07:00	Spin 4x4	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-16 16:30	Spin 4x4	Dragvoll idrettssenter	Spinningsal	Gruppetime
2026-03-16 16:30	Spin45	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-16 17:40	Spin 8x3	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-16 19:00	Spin60	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-17 07:00	Spin 8x3	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-17 18:30	Spin60	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-17 18:30	NTNUI Håndball:H1 - 2.divisjon	Dragvoll idrettssenter	Hall B	Lagtrening
2026-03-17 19:45	Spin 4x4	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-17 19:45	NTNUI Basketball:Basket	Dragvoll idrettssenter	Gymsal	Lagtrening
2026-03-18 16:15	Spin60	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-18 16:30	Spin45	Dragvoll idrettssenter	Spinningsal	Gruppetime
2026-03-18 17:30	Spin 4x4	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-18 18:30	Spin45	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-18 19:30	Spin 8x3	Øya treningssenter	Sykkelsal	Gruppetime
2026-03-18 20:30	Spin 8x3	Øya treningssenter	Sykkelsal	Gruppetime

Brukstilfelle 5:

'Lag en personlig besøkshistorie for bruker «johnny@stud.ntnu.no» siden 1. januar 2026. Denne kan lages i SQL. Sørg for at det er registrert noen treninger for Johnny i databasen. Skriv ut hvilken trening, treningssenter og dato/tid for treningen. Resultatet skal inneholde unike rader.'

Dette brukstilfellet viser tidligere registrerte treninger for en gitt bruker.

OUTPUT:

```

Velg et brukstilfelle (2-8, eller 0 for å avslutte): 5

PERSONLIG BESØKSHISTORIKK FOR EN GITT BRUKER
-----
Oppgi e-post for å hente personlig besøkshistorikk:
E-post (trykk Enter for johnny@stud.ntnu.no):

PERSONLIG BESØKSHISTORIKK FOR johnny@stud.ntnu.no
-----

```

Tid	Aktivitet	Senter	Type
2026-03-17 18:30	Spin60	Øya treningssenter	Gruppetime
2026-03-10 17:00	Spin45	Øya treningssenter	Gruppetime
2026-02-20 20:00	Spin 4x4	Dragvoll idrettssenter	Gruppetime
2026-02-10 07:00	Spin60	Øya treningssenter	Gruppetime
2026-02-08 19:45	NTNUI Håndball	Dragvoll idrettssenter	Lagtrening
2026-01-30 18:00	Spin 8x3	Dragvoll idrettssenter	Gruppetime
2026-01-18 10:00	Spin45	Øya treningssenter	Gruppetime
2026-01-14 18:30	NTNUI Håndball	Dragvoll idrettssenter	Lagtrening
2026-01-10 07:00	Spin 4x4	Dragvoll idrettssenter	Gruppetime

Brukstilfelle 6:

'Svartelisting. Brukeren 'johnny@stud.ntnu.no' fikk uheldigvis tre prikker i system og skal utestenges fra elektronisk booking i 30 dager. Implementeres i Python og SQL. Dere skal sjekke at det finnes minst tre prikker innen siste 30 dager før dere svartelister.'

Dette brukstilfellet demonstrerer hvordan systemet håndterer svartelisting gjennom triggere.

OUTPUT:

```
Velg et brukstilfelle (2-8, eller 0 for å avslutte): 6

SIMULERING AV SVARTELISTING
-----
Starter simulering av svartelisting for johnny@stud.ntnu.no ...
Oppretter tre prikker for johnny@stud.ntnu.no ...

Prikker registrert for johnny@stud.ntnu.no med brukerID 10:
-----
BrukerID |      Tid
-----
10      | 2026-03-17 17:20:00
10      | 2026-03-17 17:25:00
10      | 2026-03-17 17:30:00

Forsøker å booke en gruppetime for å sjekke om svartelisting fungerer...
-----
Suksess: Bookingen ble blokkert!
Feilmelding fra trigger: Brukeren er svartelistet på grunn av 3 eller flere prikker de siste 30 dagene.

Oppryddning: 'Test-prikkene' for johnny@stud.ntnu.no er slettet.
```

Brukstilfelle 7:

'Hver måned blir personen/personene som har trent flest fellestreninger, gitt oppmerksomhet. Lag et query som finner den/de som har deltatt på flest gruppetimer en gitt måned. Det kan være flere enn en person. Denne kan lages i Python og SQL. Husk å ta med måned som parameter. Legg inn noen som har trent slik at du viser at queriet virker.'

Dette brukstilfellet identifiserer brukere med flest gjennomførte gruppetimer.

OUTPUT:

```
Velg et brukstilfelle (2-8, eller 0 for å avslutte): 7

Finn månedens medlem:
Oppgi en måned (1-12, trykk Enter for 3):

MÅNEDENS MEDLEM(MER) FOR MÅNED 03. I 2026:
-----
Fornavn      | Etternavn      | E-post              | Antall økter
-----
Adam         | A.             | bruker1@stud.ntnu.no | 7
Eirin        | E.             | bruker5@stud.ntnu.no | 7
```

Brukstilfelle 8:

'Noen forskere ønsker å finne ut om det er vanlig å trene sammen? Foreslå en måte å finne ut av dette på. For å forenkle problemet, kan dere anta at dere skal finne to studenter som trener sammen. Altså epost, epost og antall felles treninger. Skriv dette i SQL. Legg inn noen som har trent slik at du viser at queriet virker.'

Dette brukstilfellet viser per av brukere og antall felles treninger.

OUTPUT:

```
Velg et brukstilfelle (2-8, eller 0 for å avslutte): 8

FELLES TRENINGER MELLOM TO BRUKERE:
-----
E-post bruker 1 | E-post bruker 2 | Antall felles treninger
-----
bruker1@stud.ntnu.no | bruker5@stud.ntnu.no | 7
bruker1@stud.ntnu.no | bruker3@stud.ntnu.no | 5
bruker3@stud.ntnu.no | bruker5@stud.ntnu.no | 5
bruker1@stud.ntnu.no | bruker2@stud.ntnu.no | 4
bruker1@stud.ntnu.no | bruker4@stud.ntnu.no | 4
bruker2@stud.ntnu.no | bruker5@stud.ntnu.no | 4
bruker4@stud.ntnu.no | bruker5@stud.ntnu.no | 4
bruker3@stud.ntnu.no | bruker4@stud.ntnu.no | 3
bruker2@stud.ntnu.no | bruker3@stud.ntnu.no | 2
bruker2@stud.ntnu.no | bruker4@stud.ntnu.no | 2
```

(Output vil også inkludere johnny@stud.ntnu.no hvis brukstilfelle 8 er kjørt etter brukstilfelle 2 og 3)

KI-deklarasjon

- Prosjektet er utarbeidet med støtte fra kunstig intelligens. Verktøyene som er benyttet er ChatGPT, Gemini og NotebookLM.
- All kode i prosjektet er skrevet selv. KI har ikke blitt brukt til å generere ferdig kode, men som støtte i form av diskusjon, feilsøking og vurdering av løsninger.

DB1

Bruk av KI i DB1 er dokumentert i første del av prosjektet i 'TDT4145 TreningDB DB1.pdf' under kapittel *Bruk av KI – kunstig intelligens*. I DB1 ble KI i hovedsak brukt som en sparringspartner for designfasen, og kan oppsummeres slik:

- **ER-modell og normalisering:** KI ble brukt til å diskutere tolkninger av oppgaveteksten og vurdere oversettelsen fra ER-modell til relasjonsdatabaseskjema.
- **Designdiskusjoner:** ChatGPT og Gemini ble brukt til å diskutere ulike designvalg og alternative løsninger i forbindelse med ER-modellen, samt hvordan disse kunne videreføres i relasjonsdatabaseskjemaet. KI ble brukt til å presentere egne forslag, få tilbakemeldinger og deretter vurdere disse opp mot pensum før endelige valg ble gjort.
- **Kvalitetssikring:** NotebookLM ble brukt til å sammenligne egne løsninger med pensum i emnet for å sikre at normalformer og notasjonsstandarder ble fulgt i henhold til emnets krav.

Erfaringen fra DB1 er at KI fungerte godt som et diskusjonsverktøy i tidlig designfase, spesielt for å utforske ulike tolkninger av oppgaveteksten og mulige strukturelle løsninger. Den endelige ER-modellen og relasjonsdatabaseskjemaet er likevel basert på egne vurderinger.

DB2

I DB2 har KI blitt brukt som en teknisk diskusjonspartner i arbeidet med å videreføre designet fra DB1 til en fungerende databaseapplikasjon implementert med Python og SQL. Målet har vært å støtte utviklingen av en løsning som både ivaretar integritetsreglene og forretningslogikken i oppgaven og er etterprøvable gjennom kjøring av brukstilfellene.

I DB2 ble KI hovedsakelig brukt til:

- **Tolkning av brukstilfeller:** Diskusjon av hvilke data, parametere og output-format hvert brukstilfelle krever, samt hvordan dette kan implementeres på en konsistent og korrekt måte i forhold til datamodellen og oppgaveteksten.
- **Vurdering av implementasjonsvalg:** Sammenligning av ulike løsninger, blant annet for håndtering av forretningsregler i Python sammenlignet med SQL-triggere. Dette inkluderte vurderinger knyttet til tidsavhengige regler, svartelisting og validering av input.
- **Etterprøvbarehet:** Diskusjoner rundt hvordan tidsavhengige regler kan reproduseres, som førte til bruk av simulert systemtid i databasen for å sikre konsistente testresultater uavhengig av når programmet kjøres.
- **Feilsøking og kvalitetssikring:** Analysere feilmeldinger (f.eks. SQL-syntaksfeil, fremmednøkkelbrudd og triggerkonflikter), hvor KI ble brukt til å foreslå mulige årsaker som deretter ble kontrollert og verifisert i egen implementasjon.

I DB2 har KI i større grad fungert som en teknisk assistent under implementasjonen, særlig i situasjoner hvor utviklingen har stoppet opp eller feilsøking har vært vanskelig, som er en tydelig endring fra DB1.

Erfaringer med bruk av KI

Erfaringen fra prosjektet er at KI fungerer best som et støtte- og diskusjonsverktøy, og ikke som en erstatning for faglig vurdering.

Positive erfaringer:

- KI var nyttig i utviklingsprosessen ved å gi tilbakemeldinger på egne forslag og bidra til å avklare hvordan løsninger kunne implementeres i praksis.
- KI var spesielt nyttig i feilsøking, der konkrete feilmeldinger kunne analyseres raskt og gi indikasjoner på mulige årsaker.
- KI bidro i diskusjoner rundt disse alternative implementasjonsvalg og konsekvenser av disse.
- KI hjalp spesielt når implementasjonen stoppet opp, ved å foreslå mulige feilkilder og gi retning videre i arbeidet, men uten å gi ferdige løsninger.

Begrensninger:

- Forslagene var ikke alltid i samsvar med oppgaveteksten, datamodellen eller emnets forventninger.
- KI kunne foreslå løsninger som ikke passet med eksisterende implementasjon eller skjema, for eksempel feil kobling mellom tabeller eller ikke-eksisterende felt.
- I lengre diskusjoner var det nødvendig å gjenta kontekst og forutsetninger for å få relevante og korrekte forslag.
- Det var derfor nødvendig å ha god kontroll på egen implementasjon og bruke pensum aktivt for å vurdere forslag kritisk.

Samlet sett har KI vært mest nyttig som et verktøy for diskusjon, feilsøking og kvalitetssikring. Siden jeg har gjennomført prosjektet individuelt, har KI i stor grad fungert som en erstatning for en faglig diskusjonspartner i utviklingsprosessen. Dette har vært svært nyttig i vurdering av ulike design- og implementasjonsvalg, hvor alternative løsninger kunne diskuteres og vurderes opp mot hverandre før endelige beslutninger ble tatt.

KI har bidratt til å støtte utviklingsprosessen ved å gi innspill, stille oppfølgingsspørsmål og peke på mulige konsekvenser av ulike valg. Den endelige løsningen er basert på egne vurderinger, testing og tilpasninger i tråd med oppgaveteksten, datamodell og pensum.